

[SENIOR SWE INTERVIEW PREP]

System Design Masterclass: WhatsApp

Architecting for 2 Billion Users &
100 Billion Messages Daily.

A visual playbook for structuring massive-scale system design responses.

The Challenge

Design a globally scaled, highly available, and secure messaging application capable of handling persistent connections for billions of users with strict low-latency requirements.

2 Billion
Active Users

100 Billion
Messages/Day

19.4 Hours/Month
Average Usage



Defining the Requirements (The Scope)

Functional Requirements



Conversation: 1-on-1 & Group Chats.



Acks: Delivery status (Sent, Delivered, Read).



Media: Images, Videos, Audio, Documents.



Offline Storage: Persist messages for offline users.



Push: Notify offline users upon reconnect.

Non-Functional Requirements



Low Latency: Real-time delivery with minimal delay.



Consistency: Strict message ordering & chat history sync.



Availability: High uptime prioritization.



Security: End-to-end (E2E) encryption.



Scalability: Graceful horizontal scaling.

Capacity Planning Dashboard (Proving the Scale)

Storage

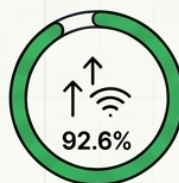


300 TB / Month

100B msgs/day x 100 Bytes = 10 TB/day

Retain undelivered messages for 30 days max.

Bandwidth

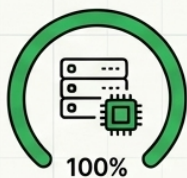


~926 Mb/s

(Ingress & Egress)

10 TB / 86400 sec = 926 Mb/s

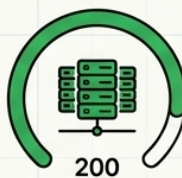
Compute Density



10M
Connections / Server

Achieved via extensive OS kernel & networking library tuning.

Server Fleet



200
Chat Servers

2B DAU / 10M per server = 200 Servers

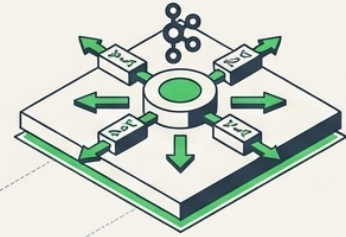
Core Building Blocks (The Toolbelt)



Mnesia Database
Message queueing (FIFO processing for offline durability).



Redis Cluster
Fast caching (Mapping users to active WebSocket servers).



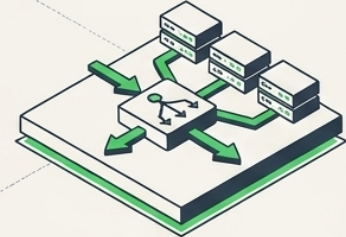
Kafka
Messaging queue (Pub/Sub for group message fan-out).



Blob Storage
Immutable storage for heavy media payloads.

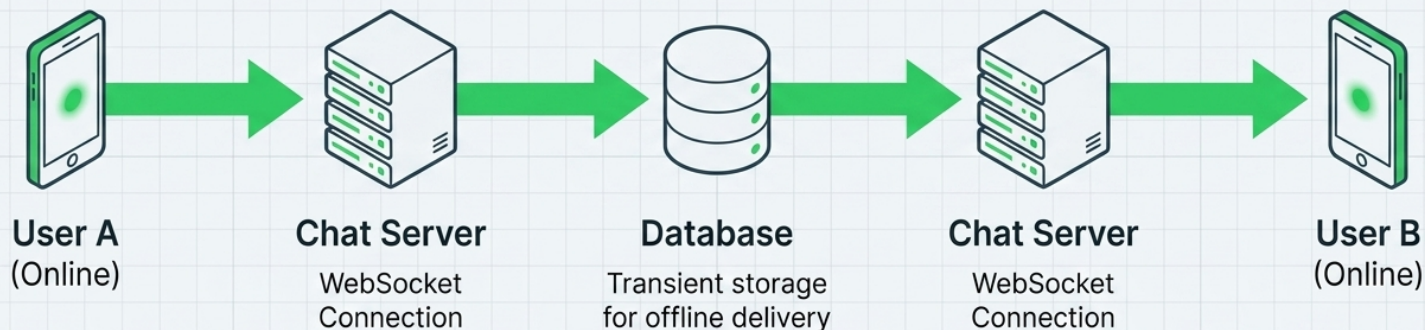


CDN
Edge delivery for frequently accessed multimedia.



Load Balancers
Traffic distribution across 200+ WebSocket servers.

High-Level Architecture (The 10,000-Foot View)



Why WebSockets? Bidirectional, persistent connections provide real-time, low-latency communication, drastically reducing overhead compared to standard HTTP polling.

Core API Design

// Sending Messages

sendMessage(sender_ID, receiver_ID, type, text, media_object, document)

The 'type' parameter determines if the payload is text (routed via WebSocket) or media (routed via Asset Service).

// Retrieving Messages

getMessage(user_ID)

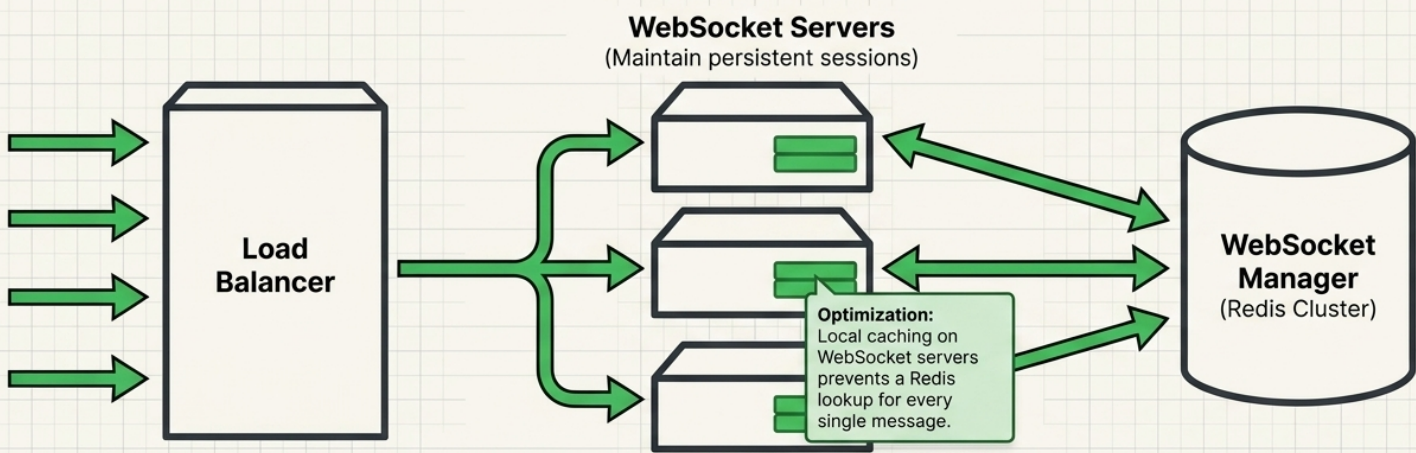
Called when an offline user reconnects to fetch the queued 30-day Mnesia backlog.

// Media Handling

uploadFile(file_type, file) -> returns file_id

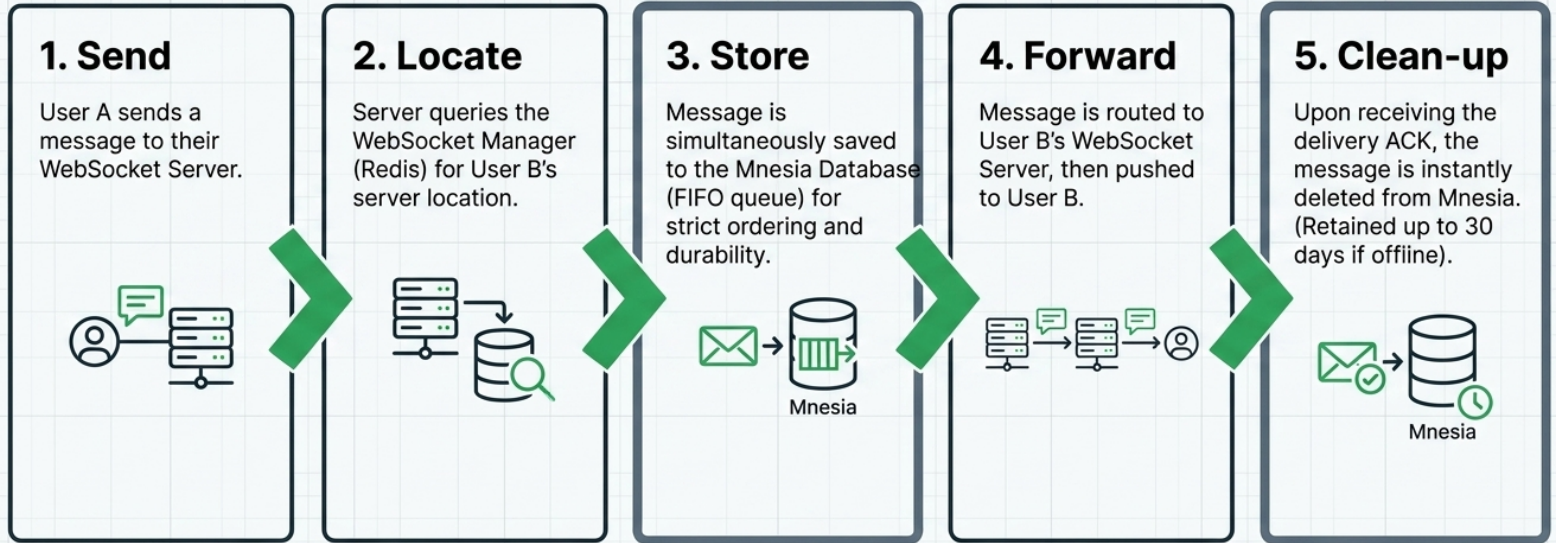
downloadFile(user_id, file_id)

Deep Dive 1: Connection Management

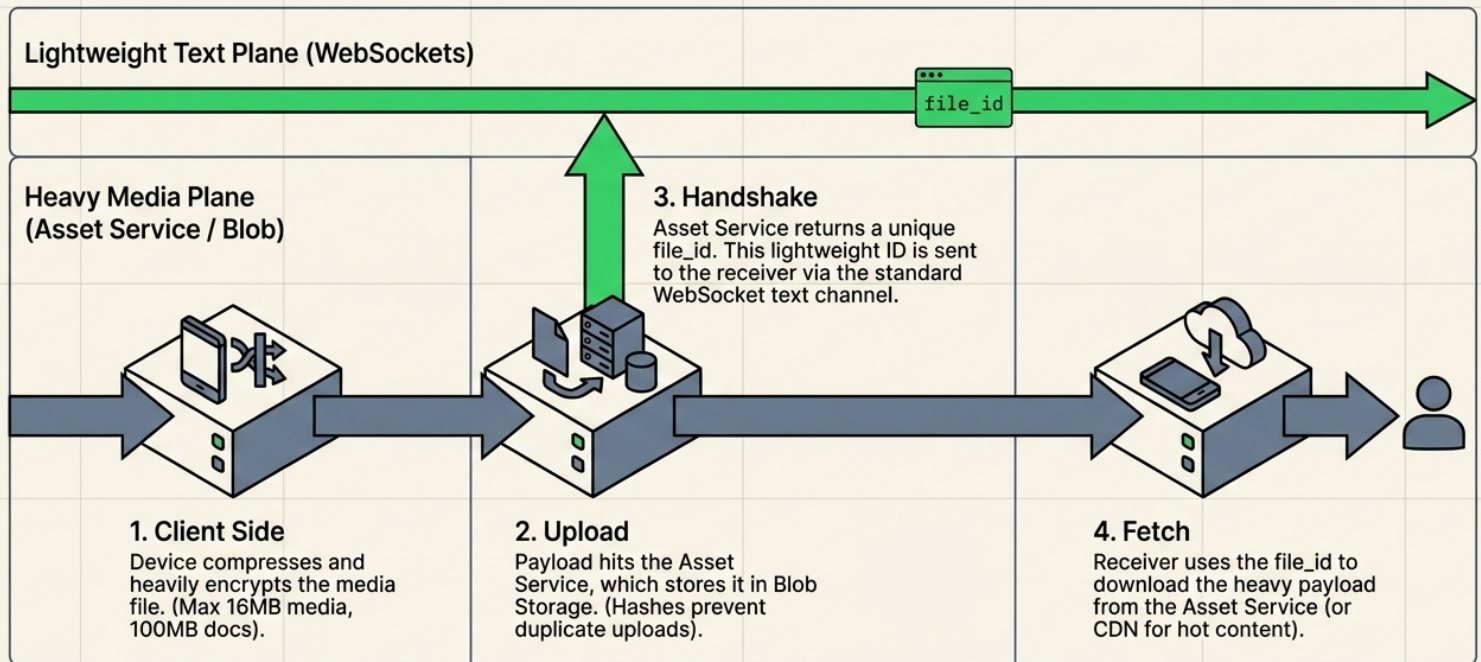


The Problem	The Solution
2 Billion users spread across 200+ servers. How does Server A find User B?	Redis maintains a real-time mapping of User ID -> Active Connection -> Specific WS Server.

Deep Dive 2: The Life of a 1-on-1 Message

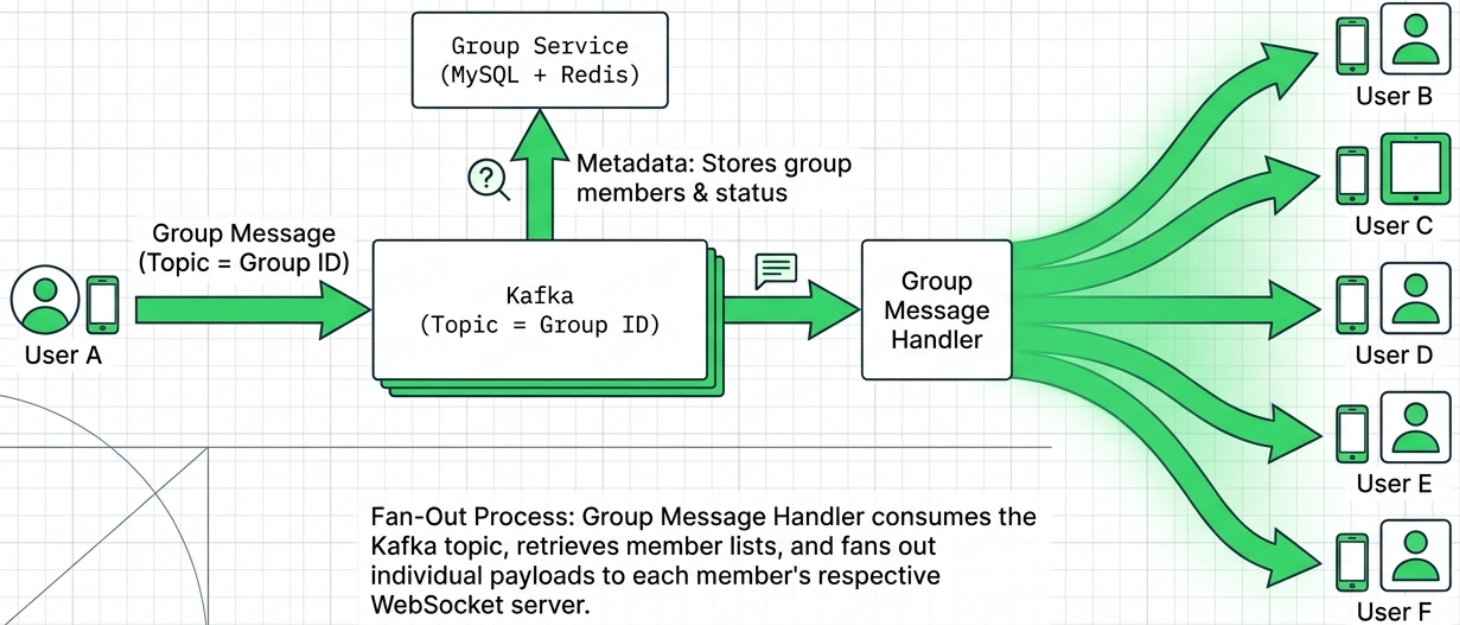


Deep Dive 3: Handling Heavy Media

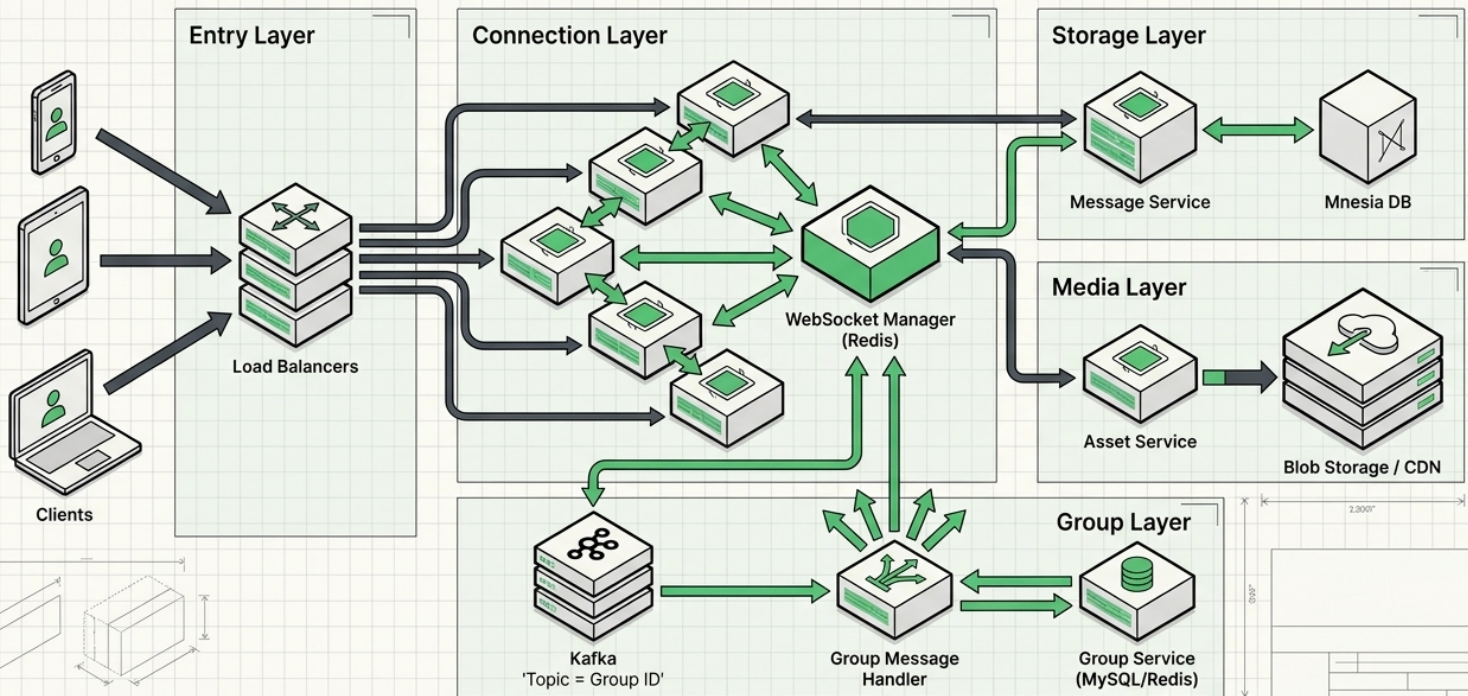


Deep Dive 4: Scaling Group Chat

The Limitation: WebSockets track active users, not groups.



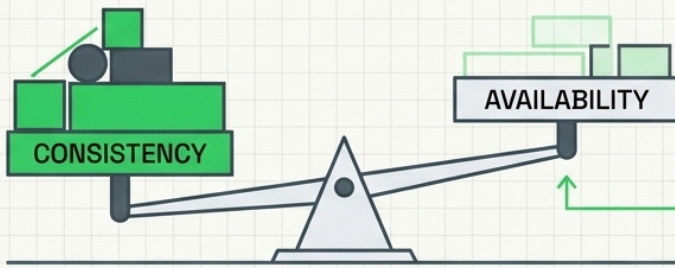
The Unified Architecture



Requirements Compliance Check

Requirement		Architectural Mechanism
✓	Low Latency	Geo-distributed WebSocket servers, local caching, and CDNs for media.
✓	Consistency	FIFO messaging queues (Mnesia) and Sequencer IDs for strict ordering/causality.
✓	Availability	Primary-secondary replicated databases; Load balancers auto-reconnect failed WS sessions.
✓	Scalability	OS/Kernel performance tuning (10M connections/server) and horizontal service scaling.

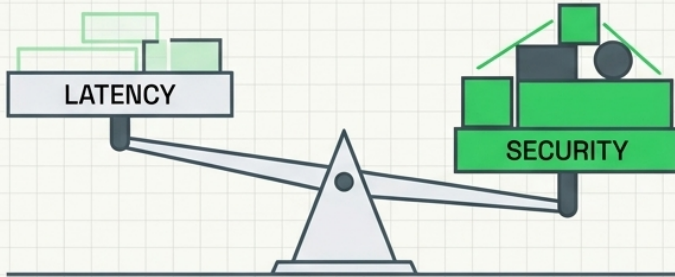
Engineering Trade-offs (The Final Test)



CAP Theorem: Consistency vs. Availability

Decision: Consistency over Availability.

Rationale: In a chat app, message ordering is paramount. In a network partition, it is better to delay a message than to deliver a conversation out of order and break causality.



Performance: Latency vs. Security

Decision: Security over theoretical minimum latency.

Rationale: End-to-end (E2E) encryption introduces heavy computational overhead (CPU limits on devices, especially when encrypting large multimedia). We accept this latency penalty as mandatory for user privacy.

Interview Recap / Takeaways

1. Justify with Math

Start with capacity planning (Storage, Bandwidth, Connections). Proving the scale constraints makes the subsequent database and load balancing choices undeniable.

2. Decouple Data Planes

Isolate lightweight, real-time flows (WebSockets/Text) from heavy, asynchronous payloads (Asset Service/Media/Kafka Groups) to prevent system bottlenecks.

3. Own the Trade-offs

No architecture is perfect. Explicitly stating that E2E encryption causes latency, or that strict message ordering sacrifices some availaby, demonstrates mature engineering judgment.

// End of Session. ■